ESCOLA
POLITÈCNICA SUPERIOR
UNIVERSITAT DE LLEIDA

Project 1: Sensor Data Extraction and Socket Communication

Programming and Communications 3

Degree in Electronic Engineering

Overview

It is a common practice to use devices such as Raspberries or Arduinos to keep control of different sensorised environments. These devices are commonly used to receive the data from the sensors and act as an intermediary between another device or a server in charge of processing the data and storing it.

This can be seen in Figure 1, where there is a wheat field with multiple sensors connected to an intermediary device in charge of collecting the data measured by the sensors, this data will be sent to a final device, server or cloud service that will process the data if required and stored.

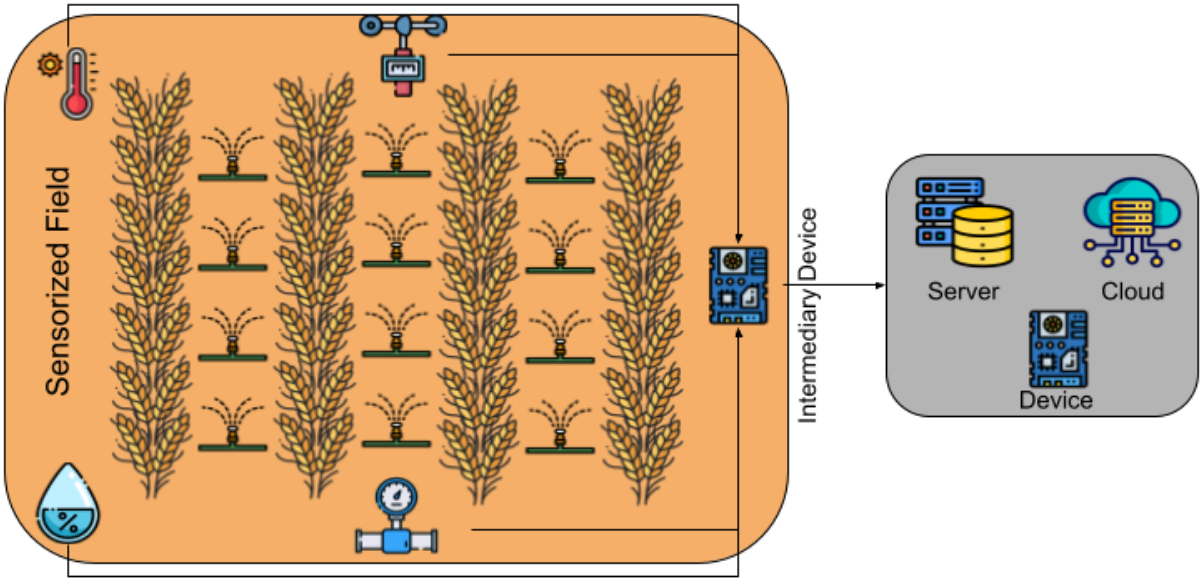

Figure 1: Sensorized field connected to an intermediary device connected to a final endpoint.

TODO

As seen in the Overview section we will create a simulation of a sensorized farm field. This field will contain 4 different sensors, Temperature, Rain Gauge, Humidity and Irrigation. In this first project, the tasks to resolve will be:

- Create a mock of each different sensor
- Code a simulation of each sensor, create data every X seconds and send it to a server to be processed.
- Create the pipeline using sockets to send the sensor data.
- Store the data in a readable JSON format.

Task 1

The first task consists of creating a mock of our sensors in the field. You will need to create the different sensors using python.

The fields that each sensor must have are the following:

- Temperature
  - Temperature: Float number, 2 decimals max
  - Timestamp: Datetime, YYYY-mm-ddTHH:MM:SS format
- Humidity
  - Humidity: Float number, 2 decimals max
  - Timestamp: Datetime, YYYY-mm-ddTHH:MM:SS format
- Irrigation:
  - Water Flow: Float number, 2 decimals max
  - Timestamp: Datetime, YYYY-mm-ddTHH:MM:SS format
- Rain Gauge:
  - Precipitation: Float number, 2 decimals max
  - Timestamp: Datetime, YYYY-mm-ddTHH:MM:SS format

Task 2

The second task consists of coding a timed process, where every 3 seconds data will be generated by each sensor as mentioned in Task 1. This data will be sent using standard python sockets

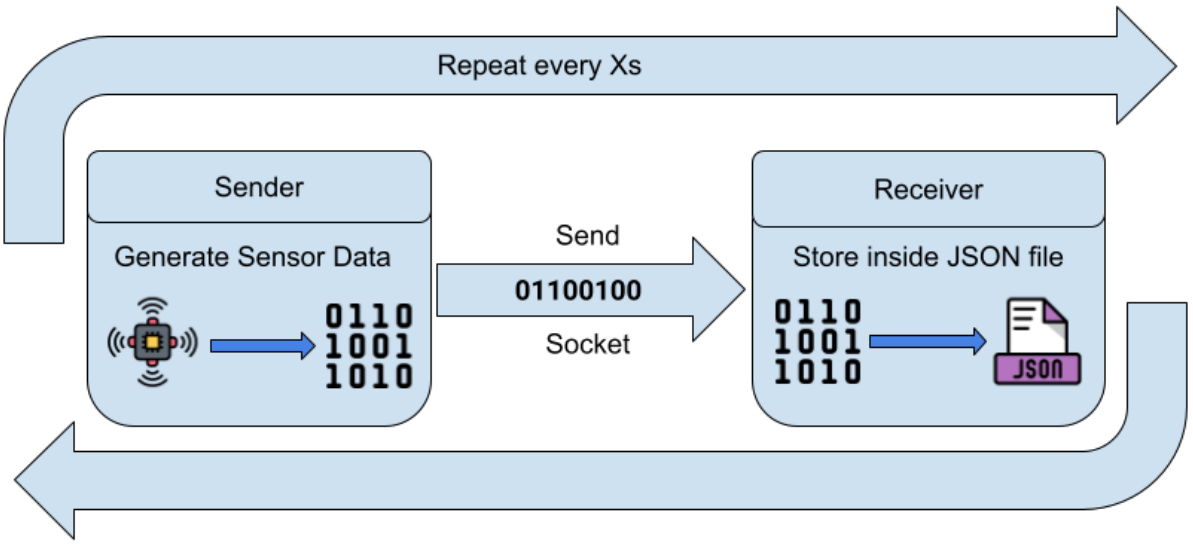

Figure 3: Behaviour of the intermediary device with the final device

Task 3

The final device needs to store the data, as we do not have any database, we will make one based on json files. Everytime the final device receives data, it will write this data into a json file, the name of the json file must be the date and the time of the received data in the format %Y-%m-%d.json

The meaning of the characters for the json name file are the following:

Symbol	Description
%Y	Year, four digits (2023)
%m	Month as a number 01-12, January is 01
%d	Day of month 01-31 (always 2 digits)
%H	Hour 00-23 (24 hour format)
%M	Minute 00-59
%S	Second 00-59

Table 1: Description of the symbols for the date string

The data must be easy to identify, therefore every sensor must have its own json file. Inside each sensor file you must store the data from that day, this means you should have a JSON file per sensor and per day.

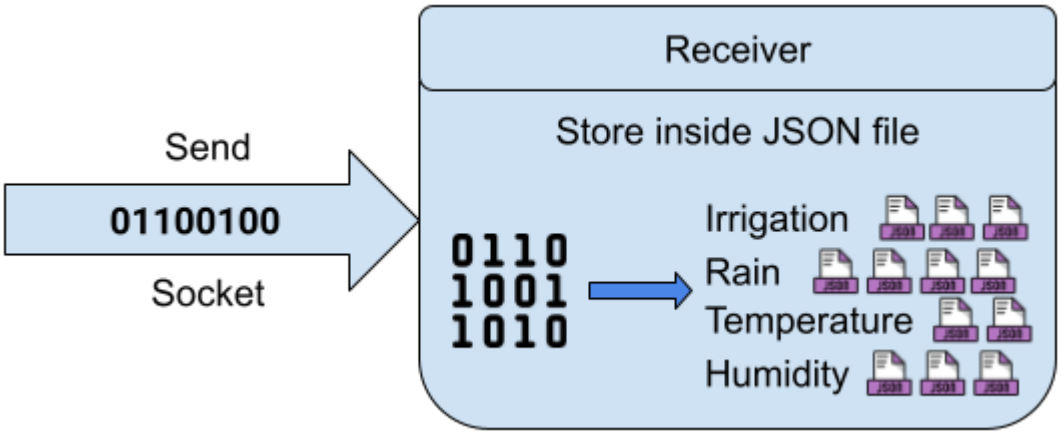

Figure 4: Behaviour of the final device after receiving data

Delivery

The project will be delivered over the virtual campus of the subject.
The content to deliver is:

- All the code related with tasks 1, 2 and 3
- A document explaining the meaningful parts of your code as well as your decisions over it.
- A link to the githubs project repository
- The document must contain all resources used

Considerations

- The quality of the code will be taken into account at the time of correcting the project.
- If the documentation does not agree with the reality of your project your mark can be reduced to 0.
- All the members of the group (if it is the case) must have commits on the githubs project repository.
- The username of the members of the group must be informed somewhere (documentation, readme in repository, …)

Resources

- Use the DateTime Library to work with dates and datetimes.
- You can use the Random Library to generate the values.
- Use the JSON Library to write data to a JSON file.